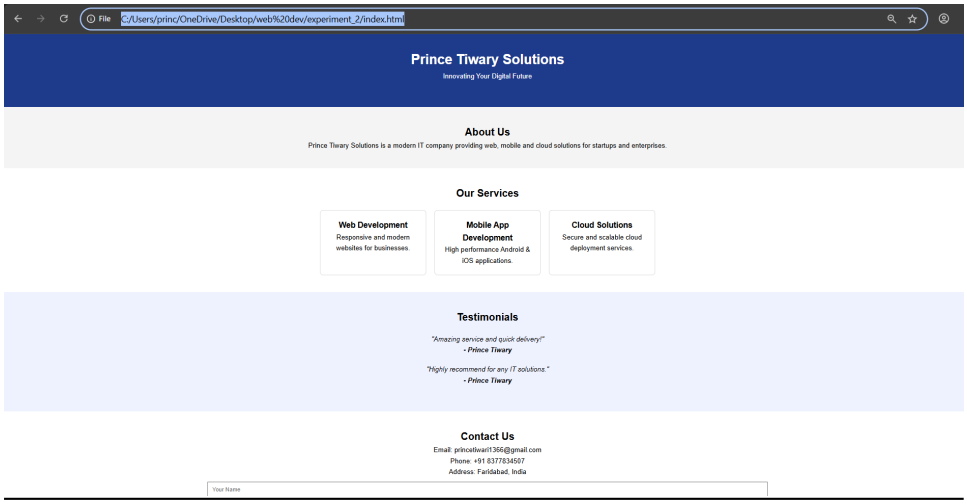
style.css

experiment_2 > style.css > *

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    line-height: 1.6;
}

.hero {
    background: #1e3a8a;
    color: white;
    text-align: center;
    padding: 60px 20px;
}

section {
    padding: 40px;
    text-align: center;
}

#about {
    background: #f4f4f4;
}

.service-container {
    display: flex;
    justify-content: center;
    gap: 20px;
    margin-top: 20px;
}

.card {
    background: white;
    padding: 20px;
    width: 250px;
    border: 1px solid #ddd;
    border-radius: 8px;
}

#testimonials {
    background: #eef2ff;
```

OUTPUT



EXPERIMENT 3

AIM:

To design and develop an **Interactive Quiz Web Application** using HTML, CSS and JavaScript that displays questions, tracks score, shows results and allows restarting the quiz.

TASKS:

. Create a project folder named **interactive_quiz**.

. Capture user answers and move between questions.

. Track and update quiz score dynamically.

. Display final result and feedback message.

. Add **Restart Quiz** feature to reuse the app.

TECH STACK USED:

Html, CSS, JavaScript

CODE:

index.html

experiment_3 > index.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Interactive Quiz</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

<div class="container">
    <h1>General Knowledge Quiz</h1>
    <p>Test your basic knowledge with this quick quiz.</p>

    <div id="quiz-box">
        <h2 id="question">Question here</h2>
        <div id="options"></div>
        <button id="nextBtn">Next</button>
    </div>

    <div id="result-box" class="hide">
        <h2 id="score"></h2>
        <p id="message"></p>
        <button onclick="restartQuiz()">Restart Quiz</button>
    </div>
</div>

<script src="script.js"></script>
</body>
</html>
```

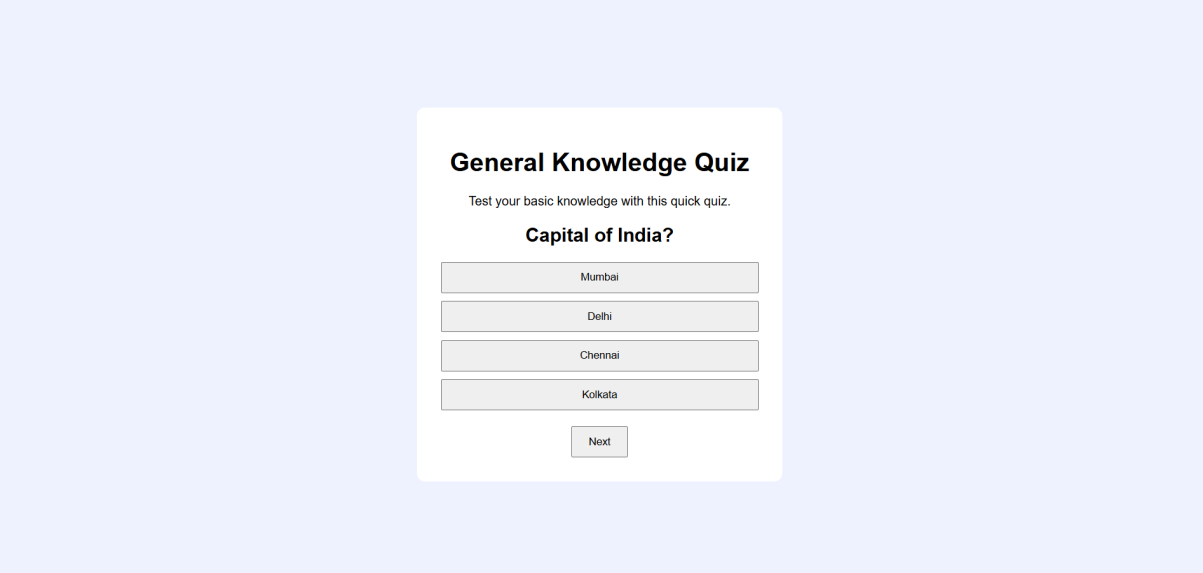
style.css

experiment_3 > style.css > body

```css
body {
    font-family: Arial;
    background: #eef2ff;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

.container {
    background: white;
    padding: 30px;
    width: 400px;
    text-align: center;
    border-radius: 10px;
}

#options button {
    display: block;
    width: 100%;
    margin: 10px 0;
    padding: 10px;
    cursor: pointer;
}

#nextBtn {
    margin-top: 10px;
    padding: 10px 20px;
}

.hide {
    display: none;
}
```

```
JS script.js
experiment_3 > JS script.js > ...
const questions = [
    {
        question: "Capital of India?",
        options: ["Mumbai", "Delhi", "Chennai", "Kolkata"],
        answer: "Delhi"
    },
    {
        question: "2 + 2 = ?",
        options: ["3", "4", "5", "6"],
        answer: "4"
    },
    {
        question: "HTML stands for?",
        options: ["Hyper Trainer Marking Language", "Hyper Text Markup Language", "High Text Machine Language", "None"],
        answer: "Hyper Text Markup Language"
    }
];

let currentQ = 0;
let score = 0;

const questionEl = document.getElementById("question");
const optionsEl = document.getElementById("options");
const nextBtn = document.getElementById("nextBtn");

function loadQuestion() {
    optionsEl.innerHTML = "";
    let q = questions[currentQ];
    questionEl.textContent = q.question;

    q.options.forEach(option => {
        let btn = document.createElement("button");
        btn.textContent = option;
        btn.onclick = () => selectAnswer(option);
        optionsEl.appendChild(btn);
    });
}

function selectAnswer(option) {
    if (option === questions[currentQ].answer) score++;
    currentQ++;

    if (currentQ < questions.length) {
        loadQuestion();
```

OUTPUT:



EXPERIMENT 4

AIM:

To design and develop a Dynamic To-Do List Web Application that allows users to add, edit, delete, and mark tasks as completed using HTML, CSS, and JavaScript.

TASKS:

. Created project folder dynamic _to do _list

. Designed To-Do interface using HTML

. Implemented dynamic task addition using JavaScript

TECH STACK USED:

HTML,CSS ,JAVASCRIPT

CODE:

index.html

experiment_4 > index.html > ...

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Dynamic To Do List</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
    <h1>Dynamic To-Do List</h1>
    <p class="desc">Add, edit, delete and complete your daily tasks.</p>

    <div class="input-section">
        <input type="text" id="taskInput" placeholder="Enter a task">
        <button onclick="addTask()">Add Task</button>
    </div>

    <ul id="taskList"></ul>

    <div class="counter">
        Completed: <span id="completed">0</span> |
        Pending: <span id="pending">0</span>
    </div>
</div>

<script src="script.js"></script>
</body>
</html>
```

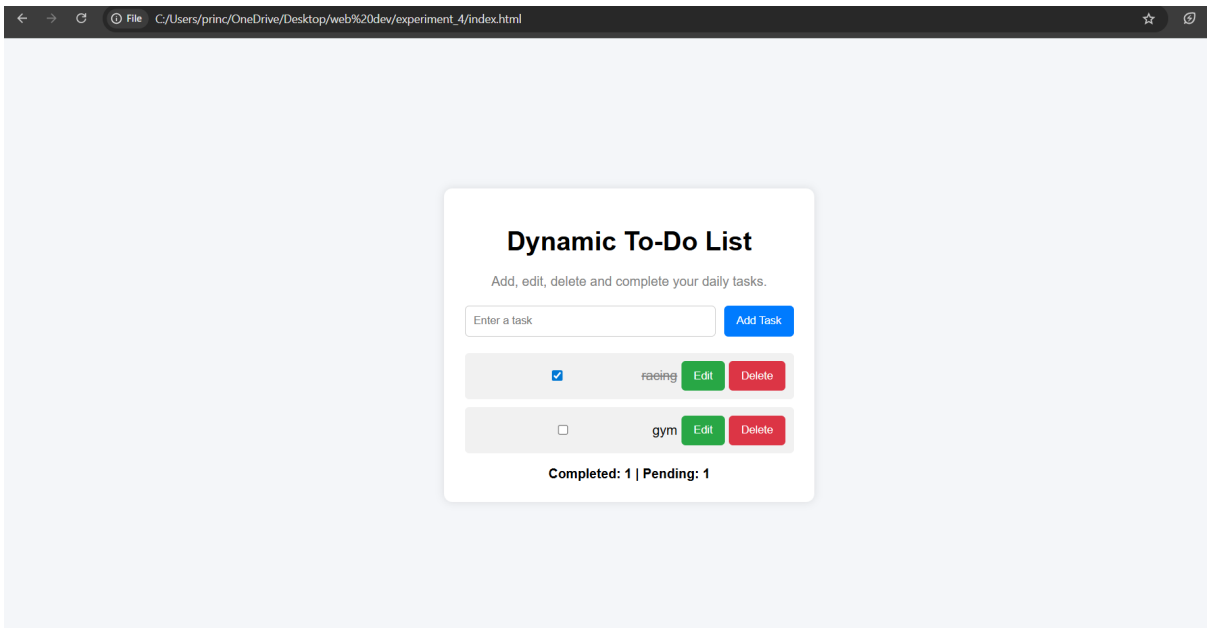
style.css

experiment_4 > style.css > body

```css
body{
    font-family: Arial, sans-serif;
    background: #f4f6f9;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}

.container{
    background: white;
    padding:25px;
    width:400px;
    border-radius:10px;
    box-shadow:0 0 10px rgba(0,0,0,0.1);
}

h1{
    text-align:center;
}

.desc{
    text-align:center;
    color: gray;
    margin-bottom:20px;
}

.input-section{
    display:flex;
    gap:10px;
}

input{
    flex:1;
    padding:10px;
    border:1px solid #ccc;
    border-radius:5px;
}

button{
    padding:10px 15px;
    border:none;
    background: #007bff;
```

```javascript
let taskList = document.getElementById("taskList");
let completedCount = document.getElementById("completed");
let pendingCount = document.getElementById("pending");

function updateCounter(){
    let tasks = document.querySelectorAll("li");
    let completed = document.querySelectorAll(".completed");
    completedCount.textContent = completed.length;
    pendingCount.textContent = tasks.length - completed.length;
}

function addTask(){
    let input = document.getElementById("taskInput");
    let taskText = input.value.trim();

    if(taskText === "") return;

    let li = document.createElement("li");

    let checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.onclick = function(){
        li.classList.toggle("completed");
        updateCounter();
    };

    let span = document.createElement("span");
    span.textContent = taskText;

    let btnDiv = document.createElement("div");
    btnDiv.className = "task-btns";

    let editBtn = document.createElement("button");
    editBtn.textContent = "Edit";
    editBtn.onclick = function(){
        let newTask = prompt("Edit task:", span.textContent);
        if(newTask !== null && newTask.trim() !== ""){
            span.textContent = newTask;
        }
    };

    let deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.className = "delete";
```

OUTPUT:



EXPERIMENT 5

AIM:

To design and develop a Simple Blog Web Application using the Flask framework that performs CRUD operations (Create, Read, Update, Delete) without using a database, and demonstrates the use of templates, routing, and basic styling in web development.

TASK :

Project Setup & Environment

Flask App Initialization

Create Operation

Read Operation

Tech Stack Used:

PYTHON, FLASK, HTML,CSS

CODE:

app.py > ...

```python

from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# Temporary storage (No database used)
posts = []

# HOME - Display all posts
@app.route('/')
def index():
    return render_template('index.html', posts=posts)

# CREATE - Add new post
@app.route('/create', methods=['GET', 'POST'])
def create():
    if request.method == 'POST':
        title = request.form['title']
        content = request.form['content']

        posts.append({
            'id': len(posts),
```

templates > index.html > ...

```html
{% extends 'base.html' %}

{% block content %}

<h2>📰 Latest Blog Posts</h2>

{% if posts %}
    {% for post in posts %}
    <div class="post">
        <h3>{{ post.title }}</h3>
        <p>{{ post.content }}</p>

        <div class="actions">
            <a href="/edit/{{ post.id }}" class="edit">Edit</a>
            <a href="/delete/{{ post.id }}" class="delete">Delete</a>
        </div>
    </div>
    {% endfor %}
{% else %}
    <p class="empty">No posts yet. Create your first post 🚀 </p>
{% endif %}
```

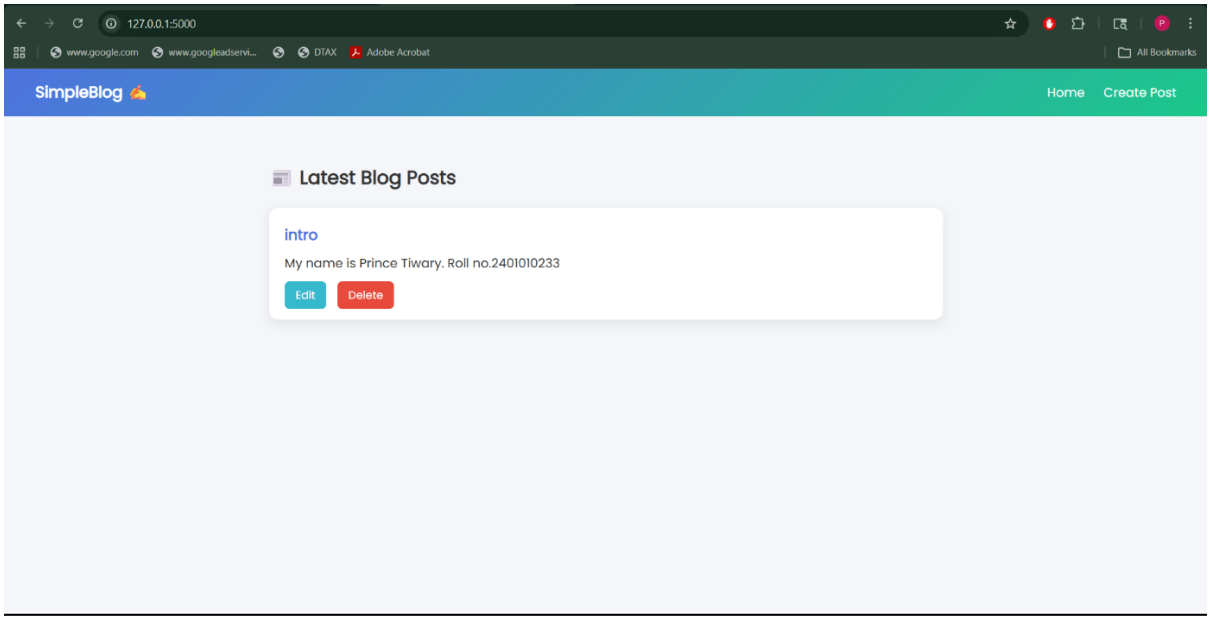
```html
{% extends 'base.html' %}

{% block content %}

<h2>Create New Post ✨</h2>

<form method="POST" class="form-card">
    <input type="text" name="title" pla

    <textarea name="content" placeholde

    <button type="submit">Publish Post</b
</form>

{% endblock %}
```

```html
{% extends 'base.html' %}

{% block content %}

<h2>Edit Post 🖍</h2>

<form method="POST" class="form-card">
    <input type="text" name="title" val

    <textarea name="content" required>{{

    <button type="submit">Update Post</b
</form>

{% endblock %}
```


```html
{% extends 'base.html' %}

{% block content %}

<h2>Create New Post ✨</h2>

<form method="POST" class="form-card">
    <input type="text" name="title" pla

    <textarea name="content" placeholde

    <button type="submit">Publish Post</b
</form>

{% endblock %}
```

```html
{% extends 'base.html' %}

{% block content %}

<h2>Edit Post 🖍</h2>

<form method="POST" class="form-card">
    <input type="text" name="title" val

    <textarea name="content" required>{{

    <button type="submit">Update Post</b
</form>

{% endblock %}
```


OUTPUT:


127.0.0.1:5000
www.google.com
www.googleadservi...
DTAX
Adobe Acrobat
All Bookmarks
SimpleBlog
Home
Create Post
Latest Blog Posts
intro
My name is Prince Tiwary. Roll no.2401010233
Edit
Delete